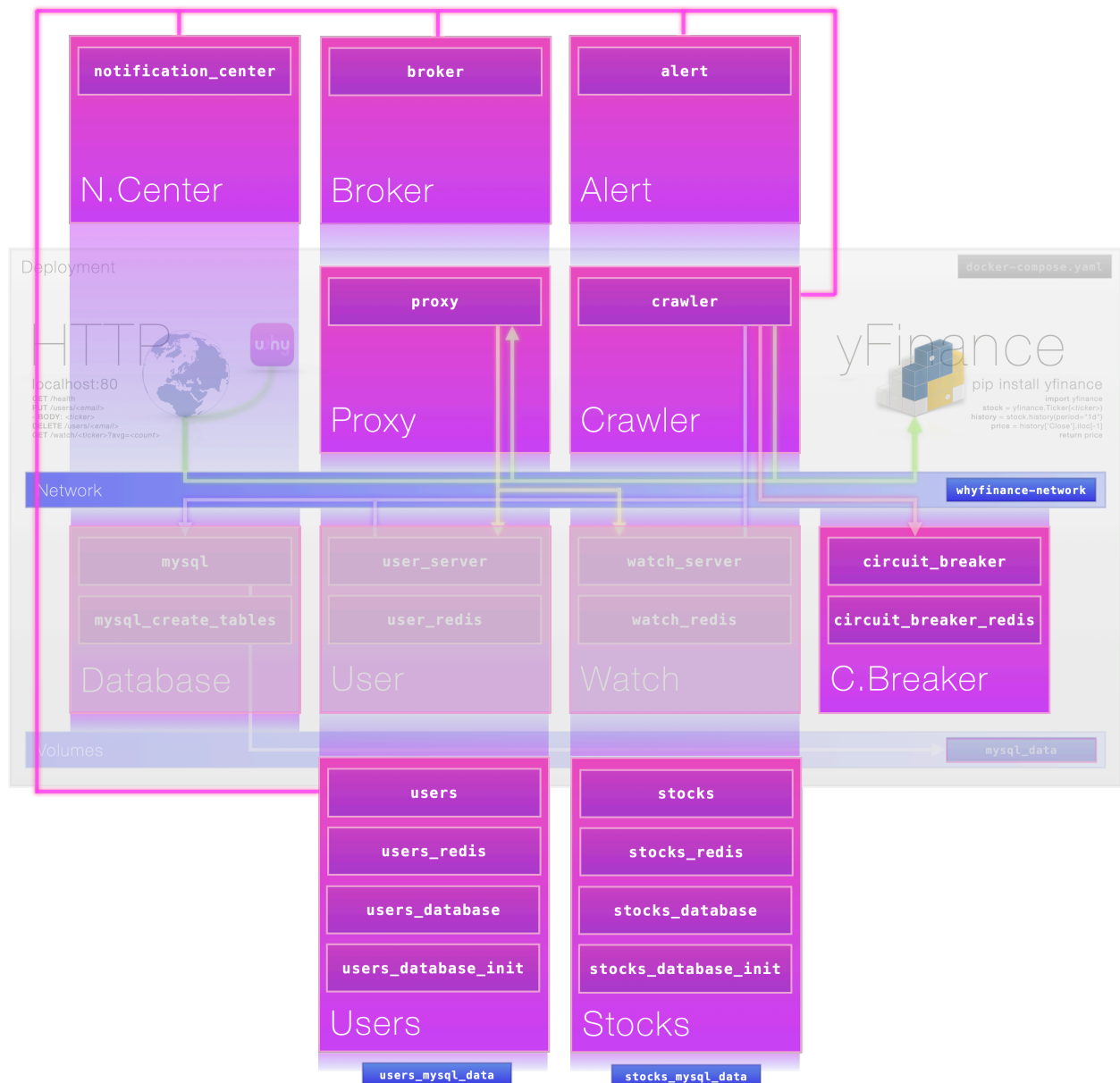


# Why Finance

## Homework 2



# Le novità



Abbiamo diviso i database, implementato il pattern CQRS e messo in comunicazione i micro-servizi aggiuntivi attraverso un singolo broker Kafka

Per la consegna di Homework 2, ispirati dal requisito circa l'implementazione del pattern CQRS che avrebbe richiesto comunque di modificare il funzionamento della parte di persistenza dei dati nel database abbiamo deciso di mettere in atto un refactoring esteso, portando a conclusione quanto avevamo iniziato nella precedente consegna e cioè la completa indipendenza tra il servizio dei dati utente, Users, e il suo database, da quello di gestione dei dati finanziari, Stocks, che adesso vive in un container separato.

## I nuovi micro-servizi e la comunicazione con Kafka

Per consentire il servizio di notifica circa il superamento di determinate soglie nel titolo di borsa scelto per l'indicizzazione, sono stati introdotti il micro-servizio **Broker**, che è un singolo broker Kafka di Confluent eseguito in modalità **KRaft** (quindi non abbiamo usato Zookeeper), senza particolari strategie di ridondanza: la complessità che ha comportato il refactoring di cui si faceva menzione prima ha purtroppo ridotto il tempo a disposizione per soluzioni più robuste.

Al broker interessano collegarsi il micro-servizio **Crawler** (che ricordiamo essere il **DataCollector** della consegna), che ad ogni indicizzazione di un titolo azionario contatta per mezzo del topic `/alert` (ovvero `/to-alert-system`) il micro-servizio **Alert** (il vostro **AlertSystem**) che per il dato titolo procede ad interrogare il database degli utenti per conoscere ove presenti le preferenze circa il recapito delle notifiche (token temporaneo del dispositivo, valore di soglia minimo e valore massimo), quindi formatta la notifica come oggetto JSON che viene consegnato, sempre per tramite Kafka e su topic `/notification-center` (il `/to-notifier`), al micro-servizio **NotificationCenter** (chiamato nel vostro testo **AlertNotifierSystem**) che lo inoltra ad Apple Push Notification Service (APNS).

In aggiunta all'uso di Kafka dettagliato nei requisiti di **Homework 2**, abbiamo anche implementato la funzionalità all'interno del micro-servizio **Users**, così che ogni qualvolta un utente si iscrive al sistema o modifica i propri dati, il valore del **ticker** da lui immesso viene immediatamente recapitato al **Crawler** su topic `/crawler` per indicizzarlo prontamente: il **Crawler** viene così modificato per usare il polling dei messaggi di Kafka al posto della banale funzione `sleep` per scandire il suo ciclo di indicizzazione, fornendo prontamente agli utenti i dati relativi a titoli di borsa che non sono già presenti nel database, senza rendere necessario allungare oltremisura il tempo di ciclo.

## Implementazione del pattern CQRS

Abbiamo adottato il pattern **Command and Query Responsibility Segregation** (CQRS) in tutti i micro-servizi che interagiscono con un database relazionale (ad esempio **Users** per la gestione degli utenti e **Stocks** per le informazioni sui titoli azionari), al fine di separare la logica di scrittura (*commands*) da quella di lettura (*queries*).

Dal punto di vista del codice, la comunicazione con il database è suddivisa in due modelli distinti: uno dedicato alle operazioni di scrittura, come inserimenti, modifiche o cancellazioni (`users_commands.py`, `stocks_commands.py`), e

uno focalizzato sulle query di sola lettura (`users_queries.py`, `stocks_queries.py`).

Anche questo è parte del refactoring di cui abbiamo già parlato, la concentrazione di tutti i sorgenti di utilità generale ai diversi container in una nuova sottocartella del progetto `/Sources`, che viene copiata in ogni container per essere sempre disponibile, senza inquinare visivamente quei sorgenti che invece sono specifici di un singolo micro-servizio, come la fornitura di un servizio gRPC.

La suddivisione fornita dal pattern CQRS si traduce in una separazione fisica e concettuale della logica di business: da un lato, i commands gestiscono transazioni e aggiornano lo stato del sistema (potendo anche includere meccanismi di transaction management, retry at-most-once o broadcast di eventi ad altri servizi, come il `Crawler`), mentre dall'altro, i queries si occupano esclusivamente di interrogare i dati (ad esempio verificare l'esistenza di un utente o recuperare informazioni su un determinato titolo), favorendo implementazioni mirate di caching o indicizzazioni senza introdurre complessità nella parte di scrittura.

In questo modo, quando un micro-servizio si avvia (tramite il proprio `main.py`), ritrova già al proprio interno i moduli dedicati sia alla scrittura che alla lettura dei dati. Il server gRPC (implementato all'interno delle classi `UserService` e `StockService` all'interno dei microservizi `Users` e `Stocks`) importa questi moduli e crea le istanze degli handler necessari: le chiamate corrispondenti a operazioni di modifica del sistema invocano i metodi di commands, mentre quelle di mera consultazione si appoggiano agli handler di queries.

Per quanto riguarda l'integrazione del pattern CQRS con il protocollo gRPC, si ha che i service method che eseguono comandi si collegano agli handler di scrittura, mentre quelli che forniscono dati richiamano direttamente gli handler di lettura.

# Guida di configurazione

## Clonare il progetto

```
git clone <https://github.com/alessiogiordano/why-finance.git>
```

## Configurazione delle Variabili d'ambiente

il file `/Containers/.env` consente di configurare in maniera centralizzata variabili d'ambiente. Ad esempio, è possibile cambiare l'intervallo di tempo tra un ciclo di aggiornamento dei dati finanziari con `CRAWLER_TIME_INTERVAL`.

Abbiamo rimosso i valori predefiniti dal `docker-compose.yml` così da prevenire l'occultamento di eventuali problemi di configurazione.

```
USERS_DB_HOST=users_database
USERS_DB_PORT=3306
USERS_DB_USER=root
USERS_DB_PASSWORD=root
USERS_DB_NAME=whyfinance_users
STOCKS_DB_HOST=stocks_database
STOCKS_DB_PORT=3306
STOCKS_DB_USER=root
STOCKS_DB_PASSWORD=root
STOCKS_DB_NAME=whyfinance_stocks

PROXY_PORT=80
REDIS_PORT=6379
WATCH_SERVER_PORT=50052
USER_SERVER_PORT=50051
CIRCUIT_BREAKER_PORT=30112

CRAWLER_TIME_INTERVAL=60 # Secondi tra un ciclo e l'altro
APNS_ENDPOINT=https://api.push.apple.com:443/3/device/
KAFKA_BROKERS=broker:9092
KAFKA_TOPICS=crawler,alert,notification-center
```

## Il nuovo script di installazione

Per guidarvi per mano nella configurazione del progetto abbiamo realizzato uno script di shell di installazione che in maniera automatica si prenderà cura di compilare i Protocol Buffer, posizzionarli nella cartella `.build/` per poi essere copiati a tempo di compilazione dei container dentro ciascuno di questi, nonché di copiare il certificato necessario per l'autenticazione presso APNS (che sarebbe inappropriato, per ragioni ovvie di sicurezza, lasciare all'interno del repository pubblico, ma che invece vi allegheremo insieme al messaggio di posta di consegna di Homework 2), oltre a avviare i vari container Docker e a consentire varie opzioni di pulizia del contesto di lavoro.

```

why-finance % ./setup.sh -h
WhyFinance
Progetto di Distributed Systems and Big Data
Anno Accademico 2024-25
(C) 2024 Luca Montera, Alessio Giordano
---
USAGE: ./setup.sh [--install-protobuf-support] [--build-protos] [--reset] [-h] ...
--transfer-certificate aps.pem      Copies the APNS certificate into the project
--install-protobuf-support         Installs the proper version of grpcio(-tools)
--build-protos                     Deletes ./Protos/.build and rebuilds ProtoBufs
--rebuild                          Forces rebuild of images before running
--reset                           Deletes built local images and volumes before running
--hard-reset                       Deletes all built images and volumes before running
--no-run                           Does not start the Docker Compose file
-h, --help                         Prints this message
why-finance %

```

Se tramite l'argomento `-h` è possibile consultare tutte le opzioni, non è comunque richiesto usarle per avviare il progetto, perché lo script verificherà autonomamente se i Protocol Buffer ed il certificato sono presenti nelle locazioni dove sono richieste, per poi mandare 'su' il `compose.yaml`.

```

why-finance % ./setup.sh --transfer-certificate ~/Desktop/aps.pem # Copierà il cert.

```

## Uso del servizio web HTTP

Il servizio HTTP alla porta 80 è largamente invariato, ad eccezione della nuova denominazione del servizio `Stocks` e della possibilità aggiuntiva di fornire i nuovi campi di User come oggetto JSON (questi campi sono assunti nulli):

```

# Restituisce 204 No Content se il sistema è in funzione
GET /health
# Prova ad effettuare l'inserimento dell'utente e se l'operazione fallisce (l'utente
esiste già) riprova aggiornando invece l'utente, restituendo 204 No Content
PUT /users/<device_id>
- BODY: <ticker> # Content-Type: text/plain
- BODY: { 'ticker':_, 'device_token':_, 'low_value':_, 'high_value':_ } # application/json
# Elimina dal database tutti i record aventi quel dato indirizzo email (per vincolo di
unicità sarà al più uno solo, ma se l'indirizzo non è presente non sarà restituito alcun
errore, ma sempre 204 No Content
DELETE /users/<device_id>
# Restituisce l'ultimo valore presente nella tabella 'stock_data' per quel dato ticker, o
sarà lanciata una eccezione se nessun dato è presente
GET /stocks/<ticker>
# Restituisce la media degli ultimi count valori per quel dato ticker, o sarà lanciata
una eccezione se nessun dato è presente
GET /stocks/<ticker>?avg=<count>

```

Non abbiamo aggiornato l'API REST per essere essa stessa oggetto di ricezione di aggiornamenti in push, quindi da questo momento in poi non vi è più parità di funzionalità tra il servizio web e l'applicazione nativa, anch'essa aggiornata per Homework 2.



# WhyFinance 2.0 per macOS e ricezione tramite Apple Push Notification Service

Abbiamo aggiornato il nostro client per macOS con una serie di miglioramenti, fra cui la possibilità di specificare opzioni circa il recapito del titolo azionario, quali il calcolo della media su più elementi, funzionalità prima non disponibile nel client nativo, ma solo tramite interfaccia REST.

Fra le nuove opzioni, ovviamente le funzionalità richieste da Homework 2, quali la possibilità di impostare una notifica al superamento di particolari soglie, che si manifestano in notifiche push di tipo `alert`.

Ogni notifica APNS è dietro le quinte un oggetto JSON trasferito verso i server Apple, e di lì nel dispositivo destinatario, attraverso HTTP2: è perfettamente

consentito in aggiunta ai campi richiesti per la presentazione della notifica, anche di trasportare altre informazioni, rendendo possibile non solo notificare l'utente del superamento della soglia, ma anche di scaricare contestualmente i nuovi dati del titolo di borsa, così che l'applicazione sia già aggiornata all'apertura.

Non solo è consentito, ma esiste un tipo di notifica alternativo ad `alert`, il `background push`, che contiene esclusivamente questo tipo di informazioni, e che non richiede permessi espliciti da parte dell'utente per essere recapitato.

WhyFinance non è al momento configurato per chiedere permesso esplicito per inviare notifiche: per impostazione predefinita saranno ricevute tutte in maniera silenziosa ai fini dell'aggiornamento dei dati dell'applicazione, ma è necessario abilitare manualmente i `banner` o gli `alert` dalle preferenze di sistema se si desidera vedere in azione su Mac l'arrivo delle notifiche.

Su dispositivi iOS è poi inoltre supportata una possibilità aggiuntiva, le notifiche provvisorie, che sono visibili nel centro notifiche, ma che non fanno suoni, e possono essere abilitate o disabilitate direttamente nella stessa schermata.

